



Analisi di mercato e del successo commerciale dei videogiochi su Steam

Alessandro Bilba, 799178
a.bilba@studenti.uniba.it

a.a. 2025/2026

link repository GitHub:

<https://github.com/albil05/Progetto-ICON-2025-2026>

Contents

1	Introduzione	2
1.1	Steam	2
1.2	Sommario	2
1.3	Strumenti utilizzati	2
2	Dataset	4
2.1	Controllo e analisi dei dati	4
2.2	Gestione dei valori nulli	5
3	Ontologia	7
3.1	Implementazione	7
3.2	Arricchimento dei dati	7
3.3	Risultati ottenuti	9
4	Apprendimento supervisionato	10
4.1	K-Nearest Neighbors (K-NN)	10
4.2	Logistic Regression	10
4.3	Decision Tree	10
4.4	Random Forest Classifier	10
4.5	Analisi degli iperparametri ottimali selezionati	10
4.6	Risultati ottenuti	11
5	Rete bayesiana	13
5.1	DAG	13
5.2	CPT	13
5.3	Simulazione degli scenari	13
5.3.1	Il blockbuster acclamato	14
5.3.2	Flop di critica	14
5.3.3	Il Free-To-Play leggero	14
5.3.4	L'Indie di nicchia a pagamento	15
5.4	Inferenza diagnostica inversa	15
5.4.1	Profilo di rischio dei giochi di successo	15
5.4.2	Probabilità di Hype tra i fallimenti commerciali	16
5.4.3	Impatto dell'hardware sul rischio	16
5.4.4	Il caso del Free-To-Play che fallisce	17
5.4.5	Conclusioni	17
6	Conclusioni e sviluppi futuri	18

1 Introduzione

1.1 Steam

Steam è una piattaforma proprietaria sviluppata da Valve Corporation che si occupa di distribuzione digitale, di gestione dei diritti digitali, di modalità di gioco multigiocatore e di comunicazione. Viene usata per gestire e distribuire una vasta gamma di giochi e il loro relativo supporto. Tutte queste operazioni sono effettuate via Internet. Ogni videogioco viene identificato tramite un identificativo numerico *AppID*. A questa chiave primaria è associato un insieme denso di attributi che ne definiscono il profilo commerciale, tecnico e sociale. Tra le feature strutturate si distinguono quelle più importanti, come il prezzo di listino e i requisiti hardware minimi di sistema (RAM, CPU), i generi di appartenenza e i tag generati dalla community, e metriche di performance, come il volume complessivo delle recensioni positive e negative. L'interazione e la combinazione di questi elementi determinano il posizionamento del prodotto all'interno del marketplace e ne influenzano in la visibilità algoritmica e il successo commerciale finale.

1.2 Sommario

Il *Knowledge-Based System* (KBS) ideato ha come obiettivo l'analisi di mercato, la valutazione del rischio e la predizione del successo commerciale dei videogiochi distribuiti sulla piattaforma, i cui dati provengono da un dataset reale composto da circa 30.000 record. Gli argomenti presi in considerazione per lo sviluppo sono stati:

1. **Ontologia astratta:** Modella la conoscenza di fondo (*Background Knowledge*) del dominio di Steam, definendo in modo astratto le relazioni tra i generi, i modelli di business e i requisiti hardware tramite standard del Web Semantico (RDF/OWL).
2. **Motore a regole:** Interroga il grafo semantico mediante query strutturate SPARQL e proietta le regole logiche deduttive (*If-Then*) sulla massa dei dati grezzi, arricchendo il dataset con nuove feature semantiche ad alto valore informativo (es. rischio commerciale e attrattività per la community).
3. **Machine Learning:** Sfrutta il dataset arricchito per addestrare e validare in modo robusto (su 10 run computazionali indipendenti) quattro algoritmi di classificazione supervisionata, dimostrando empiricamente come l'iniezione di conoscenza ontologica stabilizzi e ottimizzi le performance predittive.
4. **Rete Bayesiana:** Gestisce l'incertezza del mercato strutturando un Modello Grafico Probabilistico (PGM) ad albero causale. Attraverso tecniche di inferenza esatta, il sistema permette di effettuare analisi di scenario (*What-If Analysis*), calcolando l'esatta probabilità di successo di un prodotto al variare delle condizioni commerciali e d'uso.

1.3 Strumenti utilizzati

Per lo sviluppo, la validazione e l'esecuzione dell'intero progetto, sono stati integrati diversi strumenti. Di seguito vengono elencati i principali strumenti utilizzati:

- **Google Colab:** L'ambiente di sviluppo integrato principale basato su *Jupyter Notebook*.
- **Pandas e NumPy:** Pandas è stato utilizzato per il caricamento, la pulizia strutturale, l'estrazione dei generi e la gestione tabulare dei DataFrame (compreso il *One-Hot Encoding*). NumPy è stato impiegato per il supporto alle operazioni matematiche e per il calcolo statistico delle metriche aggregate (medie e deviazioni standard).
- **RDFLib:** E' la libreria Python fondamentale per il Web Semantico. E' stata utilizzata per definire la *TBox* dell'ontologia (classi e proprietà tramite i namespace RDF, RDFS e OWL), per popolare la *ABox* in modo dinamico e per serializzare il grafo finale nel file standard RDF/XML. E' stata sfruttata per eseguire il parsing del file e per l'interrogazione tramite query **SPARQL**.

- **Scikit-Learn (Sklearn):** È la libreria di riferimento per il Machine Learning in Python. Ha permesso la gestione dell'intera pipeline: dalla standardizzazione geometrica delle feature (**StandardScaler**), alla separazione stratificata dei campioni (**train_test_split**), fino all'importazione, addestramento e calcolo delle metriche di valutazione dei quattro modelli di classificazione testati (*K-NN*, *Logistic Regression*, *Decision Tree* e *Random Forest*).
- **Pgmpy:** E' la libreria specializzata nel calcolo e nell'apprendimento dei modelli probabilistici grafici, utilizzata sia per definire la topologia causale della rete (**DiscreteBayesianNetwork**), sia per stimare i parametri delle tabelle CPT tramite massima verosimiglianza (**MaximumLikelihoodEstimator**), sia per risolvere le query probabilistiche degli scenari tramite l'algoritmo di inferenza esatta **VariableElimination**.
- **Matplotlib e Seaborn:** Sono le due librerie per la visualizzazione dei dati. **Matplotlib** ha gestito la geometria e l'impaginazione dei layout, mentre **Seaborn** è stato utilizzato per generare grafici.

2 Dataset

Il dataset utilizzato è *SteamGames.csv*, reperibile su [Kaggle](#). Esso contiene un totale di 29779 giochi presenti sulla piattaforma Steam e un totale di 18 feature per ciascuno di esso:

- AppID, ovvero l'ID univoco assegnato a ciascun gioco.
- Name, il nome ufficiale del gioco.
- Type, ovvero la categoria dell'elemento, ad esempio gioco, DLC (contenuti aggiuntivi scaricabili) o altro.
- ReleaseDate, data di rilascio del gioco su Steam.
- Genres, il genere assegnato (Avventura, Azione, Giochi di Ruolo, ecc...).
- Developers, sviluppatori del gioco.
- Publishers, nome dell'editore del gioco.
- Description, descrizione del gioco.
- Price, prezzo del gioco. I giochi gratis sono rappresentati con uno 0.
- Thumbnail, URL dell'immagine di copertina del gioco.
- Tags, tag assegnati dalla community (shooter, free to play, third/first person e così via).
- ReviewScore, punteggio di gradimento degli utenti
- PositiveReview, numero di recensioni positive
- NegativeReview, numero di recensioni negative
- OsRequirement, requisiti di sistema operativo per il gioco.
- MemoryRequirement, RAM richiesta.
- CpuRequirement, CPU richiesta
- Rank, posizione del gioco nella classifica di Steam.

2.1 Controllo e analisi dei dati

Prima di effettuare qualsiasi operazione, vi è una fase di controllo e analisi dei dati, eventualmente per evidenziare la presenza di duplicati o valori nulli, così come la distribuzione dei dati al suo interno. Successivamente c'è stata una fase di pulizia dei dati, in cui innanzitutto sono stati modificati i valori nulli presenti. Per le variabili categoriali ad alta cardinalità come i generi, la sostituzione dei valori mancanti è avvenuta mediante l'assegnazione di una stringa vuota, evitando eccezioni durante le successive operazioni. Al fine di standardizzare i dati tecnici relativi ai requisiti hardware minimi presenti nella colonna *MemoryRequirement*, l'assenza di specifiche testuali esplicite è stata gestita attraverso una strategia di calcolo del valore basata sulla RAM mediana del segmento di mercato; a cascata, le colonne *CpuRequirement* e *OsRequirement* sono state sincronizzate attraverso la definizione di profili tecnologici coerenti basati sui gigabyte stimati (es. **High-End Modern CPU** e **Windows 10/11 64-bit** per valori ≥ 12 GB). Parallelamente, i valori mancanti legati alle metriche di performance e al prezzo di listino (*price*, *ReviewScore*, *PositiveReview*, *NegativeReview*) sono stati azzerati e convertiti rispettivamente nei tipi di dato nativi float (0.0) ed integer (0), azzerando le eccezioni di calcolo a runtime ed evitando un'eliminazione dei record che avrebbe compromesso l'integrità statistica del campione originale.

```

Tipi di dato:
Appid          int64
Name           object
Type           object
ReleaseDate    object
Genres         object
Developers     object
Publishers     object
Description    object
price          object
Thumbnail      object
Tags           object
ReviewScore    int64
PositiveReview int64
NegativeReview int64
OsRequirement  object
MemoryRequirement object
CpuRequirement object
Rank           int64

```

(a) Tipi di dati nel dataset.

```

Valori mancanti:
Appid          0
Name           0
Type           0
ReleaseDate    14
Genres         413
Developers     26
Publishers     1045
Description    52
price          0
Thumbnail      0
Tags           1614
ReviewScore    0
PositiveReview 0
NegativeReview 0
OsRequirement 2937
MemoryRequirement 3984
CpuRequirement 3859
Rank           0
dtype: int64

Duplicati completi: 0

```

(b) Duplicati e valori nulli

Figure 1: Analisi del dataset.

2.2 Gestione dei valori nulli

Le sostituzioni effettuate, nello specifico, sono state:

- **Stima della memoria RAM:** Calcolata dinamicamente tramite la mediana del segmento di mercato, incrociando il genere primario del titolo e la sua fascia di prezzo (*Price Tier*), evitando l'assegnazione statica di valori nulli.
- **Profilo CPU:** Assegnazione parametrica basata sui GB di RAM effettivi o stimati, iniettando High-End Modern CPU (≥ 12 GB), Mid-Range CPU (≥ 8 GB) o Entry-Level CPU (< 8 GB).
- **Profilo Sistema Operativo:** Sincronizzato simmetricamente con l'architettura hardware richiesta, iniettando Windows 10/11 (64-bit) per i titoli pesanti e moderni, fino a Windows 7/8/10 (64-bit) per le produzioni indipendenti o legacy.
- **Unknown** per i valori relativi ad autore, editore e tags.
- **No description** per la descrizione del gioco.
- **2026-01-01** per le date di rilascio mancanti.
- **Indie** per il genere dei giochi.

Al fine di comprendere la struttura delle variabili quantitative e identificare pattern di correlazione preliminari, si è passati ad una fase di analisi dei dati, con l'uso di visualizzazione grafica articolata in tre fasi, sfruttando le librerie *Seaborn* e *Matplotlib*.

Nello specifico, le tre analisi grafiche si focalizzano sui seguenti aspetti:

1. **Distribuzione dei prezzi:** Mostra la frequenza dei prezzi di listino dei software. Per prevenire distorsioni visive causate da un numero esiguo di *outlier* estremi (es. pacchetti software o edizioni speciali dal costo anomalo), è stato applicato un filtro booleano restrittivo che isola i soli titoli con un prezzo inferiore o uguale a 100 €. La curva KDE (*Kernel Density Estimate*) sovrapposta evidenzia un andamento fortemente asimmetrico a destra, tipico dei mercati digitali, con una concentrazione massiva di titoli nella fascia *budget* o *free-to-play*.

2. **Distribuzione del ReviewScore:** Rappresenta la frequenza dei punteggi di gradimento assegnati dalla community. All'interno del grafico è stata tracciata una linea verticale di riferimento in corrispondenza del valore critico 7.5, contrassegnata in rosso. Questa linea segna la soglia logica utilizzata successivamente dal sistema per la discretizzazione e la generazione della variabile obiettivo *Target_Success*, permettendo di valutare il bilanciamento delle classi all'interno del campione.
3. **Correlazione tra prezzo e recensioni positive:** Esplora la relazione tra il costo del gioco e il volume di recensioni positive ricevute. Data l'altissima varianza del contatore delle recensioni (il quale spazia da zero a milioni di unità, rischiando di schiacciare la quasi totalità dei punti sull'asse), sull'asse delle ordinate è stata applicata la trasformazione logaritmica non lineare $\log(x + 1)$ tramite la funzione `np.log1p`. E' possibile identificare visivamente le aree ad alta densità, indice di come la popolarità di un titolo non sia linearmente dipendente dal suo prezzo di listino.

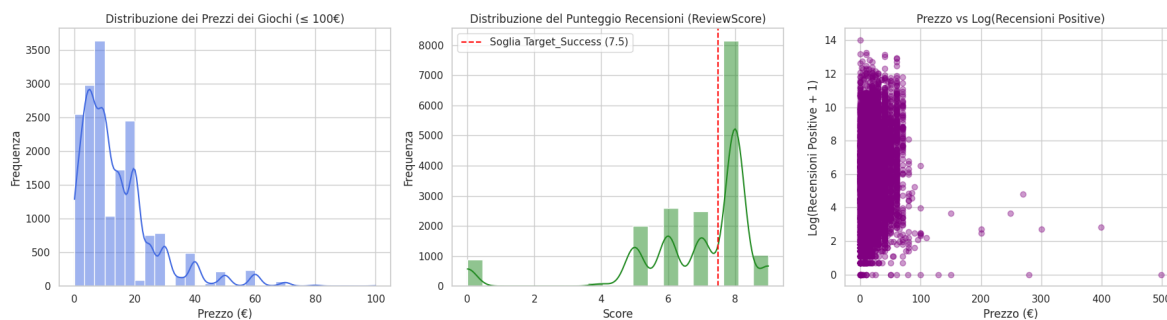


Figure 2: Grafici relativi all'analisi dei dati.

```
[STATISTICHE CHIAVE DEL DATASET]:
-> Numero totale di giochi censiti: 29931
-> Prezzo Medio di un gioco: 11.83€ (Max: 499.00€)
-> Punteggio Recensioni Medio: 6.45/10
-> Gioco più popolare (Più recensioni positive): 'Counter-Strike 2' con 1,214,897 recensioni.
```

Figure 3: Statistiche principali

3 Ontologia

Un'ontologia si definisce come una rappresentazione esplicita, formalizzata e condivisa di una concettualizzazione di un dominio di interesse. In particolare, essa ne esplicita il significato semantico e le relazioni logiche intrinseche attraverso un formalismo basato sulla logica descrittiva, strutturandosi in una componente astratta (*TBox*) e in una fattuale (*ABox*).

In questo progetto, la progettazione e l'implementazione di dell'ontologia dedicata al marketplace di Steam (espressa tramite gli standard RDF/OWL) serve a dotare il sistema di una conoscenza di sfondo (*Background Knowledge*) solida e strutturata. Questa operazione permette di superare i limiti dell'approccio puramente statistico dei dati grezzi: il sistema acquisisce la capacità di effettuare ragionamento automatico e inferenze deduttive. Ciò ha permesso di arricchire il dataset con nuove *feature* semantiche, guidando con maggiore precisione le successive fasi di classificazione predittiva e analisi probabilistica sotto incertezza.

3.1 Implementazione

Una volta conclusa la fase di pre-processing dei dati, si è proceduto all'implementazione software dell'ontologia astratta sfruttando la libreria `rdflib` in Python. L'architettura del grafo semantico è stata strutturata separando la *TBox* dall'*ABox*, inserendo l'intero vocabolario nell'*Namespace* personalizzato <http://www.semanticweb.org/uniba/icon/steam#>.

A livello di *TBox*, sono state formalizzate le classi principali destinate a mappare i concetti fondamentali del dominio: **Genere**, **ModelloBusiness** e **TierHardware**. Parallelamente, sono state dichiarate le proprietà distinguendo tra:

- *Object Properties*: relazioni logiche tra istanze di classi diverse, nello specifico `compatibileCon` (collegamento tra generi e requisiti hardware) e `modelloConsigliato` (collegamento tra generi e modelli di monetizzazione).
- *Datatype Properties*: attributi letterali atomici associati alle risorse, come `nomeGenere` mappato sul tipo di dato nativo `XSD.string`.

La fase di popolamento della *ABox* ha previsto l'inserimento delle istanze costanti di sfondo, definendo i modelli commerciali (`Biz_FreeToPlay`, `Biz_PremiumPaid`) e i profili prestazionali minimi (`Tier_HighEnd`, `Tier_Standard`). Successivamente, il sistema esegue un ciclo di estrazione sul dataset, isolando l'insieme dei generi unici per evitare ridondanze. Per ciascuno, viene generato un *URIRef* e vengono definite le relative triple logiche.

Infine, l'ontologia astratta viene arricchita applicando regole semantiche di associazione euristica: i generi afferenti al segmento "*Free*" vengono vincolati al modello di business non a pagamento, mentre le categorie che richiedono hardware più moderno (es. *Action*, *Simulation*, *Strategy*) vengono collegate al profilo hardware *High-End*. Il grafo risultante viene memorizzato in formato RDF/XML all'interno del file `steam_abstract_ontology.rdf`.

3.2 Arricchimento dei dati

Questa in pratica il motore di inferenza, combinando la conoscenza formale con i dati grezzi memorizzati nel DataFrame originale. Questo approccio combina la potenza del Web Semantico con la scalabilità delle strutture dati parallele in Python, arricchendo i record con nuove metriche semantiche molto utili. Il processo logico-computazionale si divide in tre fasi:

1. **Caricamento e interrogazione della Base di Conoscenza**: Utilizzando il linguaggio di interrogazione standardizzato SPARQL, viene eseguita una query per estrarre le relazioni associative precedentemente definite. La query estrae i generi unici, i modelli di business consigliati e i requisiti hardware associati, utilizzando clausole `OPTIONAL` per preservare la robustezza del recupero delle informazioni:

```
query_generi = """
PREFIX steam: <http://www.semanticweb.org/uniba/icon/steam#>
SELECT ?nomeGenere ?modelloBiz ?tierHardware
WHERE {
```



```

?g rdf:type steam:Genere .
?g steam:nomeGenere ?nomeGenere .
OPTIONAL { ?g steam:modelloConsigliato ?modelloBiz } .
OPTIONAL { ?g steam:compatibileCon ?tierHardware } .
}
"""

```

2. **Caching semantico:** i risultati della query SPARQL vengono deserializzati e organizzati in un dizionario di consultazione rapida denominato `kb_generi`. Questa operazione di indicizzazione mappa ogni genere testuale al suo esatto profilo ontologico (`biz_model` e `hardware_tier`), riducendo la complessità computazionale delle successive letture.
3. **Esecuzione del Motore a Regole condizionali:** il sistema esegue un ciclo iterativo sui record del dataset tramite il costrutto `for index, row in df.iterrows():`. Per ogni videogioco, vengono calcolate le informazioni atomiche (estraendo i valori numerici di `price`, `ReviewScore`, `PositiveReview`, `NegativeReview` e calcolando la variabile `Target_Success` sulla soglia ≥ 7.5) e valutate regole di produzione condizionali per inferire nuove feature categoriali, come:

- **High_Hype:** esprime il successo commerciale e la viralità di un prodotto sul mercato. A differenza di euristiche basate sul voto della critica (che genererebbero fenomeni di *Data Leakage* sulle metriche di target), questa *feature* viene asserita combinando esclusivamente criteri economici e di volume. Il valore viene impostato a 1 solo se il videogioco dimostra un grande volume di recensioni positive superiore a 5000 unità a fronte di un prezzo accessibile o competitivo, fissato sotto la soglia dei 40 €:

```
high_hype = 1 if pos_rev > 5000 and price < 40.0 else 0
```

- **Monetization_Model e Hardware_Tier:** estraggono e propagano al singolo gioco i vincoli di business e hardware ereditati dai suoi generi di appartenenza cercandoli all'interno del dizionario `kb_generi`, integrando la regola per cui se la RAM estratta è ≥ 12 GB il profilo viene forzato a `High_End_Specs`:

```
monetization_model = "Free_To_Play" if "FreeToPlay" in kb_generi[g_gioco]["biz_model"]...
if "HighEnd" in kb_generi[g_gioco]["hardware_tier"] or ram >= 12:
    hardware_tier = "High_End_Specs"
```

- **Commercial_Risk:** valuta la stabilità finanziaria del prodotto, etichettando il gioco come `Rischio_Elevato` se un prezzo di listino premium (≥ 40.0) in regime `Premium_Paid` non è supportato da una risposta soddisfacente della community (meno di 2000 recensioni positive), secondo la regola.

Il risultato finale è la generazione del dataset consolidato memorizzato in `"steam_enriched.csv"`. Questo dataset arricchito contiene non solo dati grezzi, ma anche la conoscenza dedotta dal sistema a regole, fornendo una matrice di feature ottimizzata per i successivi stadi di apprendimento supervisionato e inferenza probabilistica.

	Name	High_Hype	Monetization_Model	Hardware_Tier	Commercial_Risk	Target_Success
0	Resident Evil Requiem	0	Premium_Paid	High_End_Specs	Rischio_Controllato	1
1	Counter-Strike 2	1	Premium_Paid	High_End_Specs	Rischio_Controllato	1
2	ARC Raiders	1	Premium_Paid	High_End_Specs	Rischio_Controllato	1
3	Tom Clancy's Rainbow Six Siege	1	Premium_Paid	High_End_Specs	Rischio_Controllato	1
4	Apex Legends™	0	Premium_Paid	High_End_Specs	Rischio_Controllato	0

Figure 4: Esempio del dataset con le nuove feature

3.3 Risultati ottenuti

E' stata effettuata un'analisi tra la variabile dedotta dal motore semantico **High_Hype** e la variabile obiettivo reale del dataset **Target_Success**. L'analisi computazionale ha restituito tre output principali:

1. **Tabella di contingenza reale:** calcolata tramite l'istruzione `pd.crosstab`, mappa il conteggio assoluto dei giochi incrociando le dimensioni. Gli indici e le colonne sono stati ridenominati (**High_Hype:** NO/SÌ e **Target_Success:** NO/SÌ) per garantire leggibilità. La matrice risultante evidenzia numericamente la distribuzione delle frequenze reali e la forte asimmetria tipica del mercato digitale.

2. **Mappa di calore:** La matrice, colorata con gradazioni di viola, esplicita la densità dei dati nelle quattro celle di contingenza. L'inserimento dei valori assoluti in formato intero consente di analizzare la corrispondenza tra la deduzione logica (asse delle ordinate) e il target commerciale effettivo (asse delle ascisse).

3. **Grafico a barre:** Calcola prima le percentuali condizionate normalizzate per riga sull'asse del target:

```
tabella_percentuale = pd.crosstab(df_analysis['Target_Success'],  
                                  df_analysis['High_Hype'],  
                                  normalize='index') * 100
```

Il grafico mostra visivamente la composizione percentuale della variabile **High_Hype** all'interno delle due classi del **Target_Success** (Fallimento vs Successo).

Questa visualizzazione dimostra l'effetto discriminante della conoscenza introdotta: mentre nella classe del fallimento (**Target_Success** = 0) la quota di **High_Hype** = 1 è matematicamente trascurabile, essa si concentra prevalentemente all'interno della classe dei successi reali (**Target_Success** = 1).

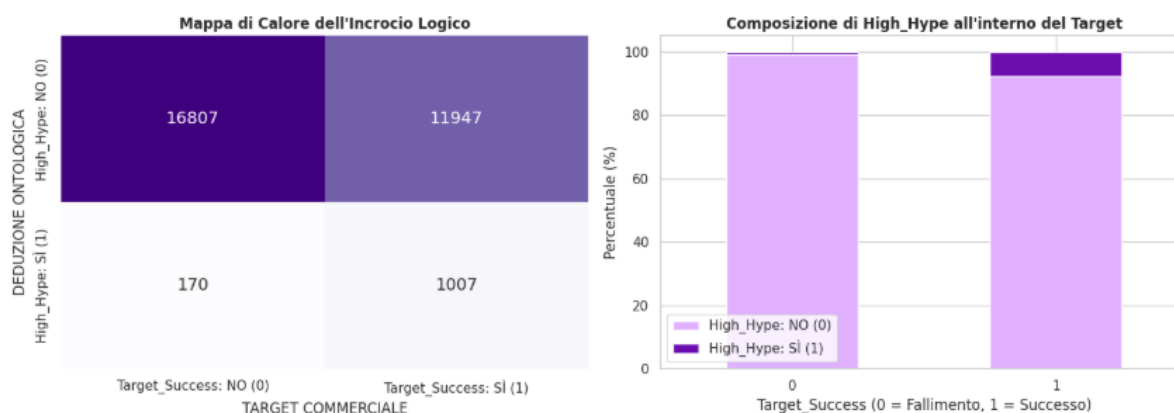


Figure 5: Grafici ottenuti

4 Apprendimento supervisionato

La successiva fase di apprendimento supervisionato consiste nell'addestramento di 4 modelli diversi per un totale di 10 run, sia sul dataset di base che su quello arricchito grazie all'uso dell'ontologia generata, al fine di confrontare gli effetti dell'arricchimento dei dati. Al fine di garantire la massima equità nel confronto, l'addestramento di ciascun modello in ogni singolo run viene preceduto da una fase di *Fine-Tuning* degli iperparametri mediante una procedura di **Grid Search** associata a una Cross-Validation a 3 fold, ottimizzando esplicitamente la metrica dell'F1-Score.

4.1 K-Nearest Neighbors (K-NN)

Il *K-Nearest Neighbors* è stato inserito all'interno della pipeline di ottimizzazione iperparametrica esplorando un range di vicini (`n_neighbors`: [3, 5, 7]). Trattandosi di un algoritmo basato sulla distanza geometrica, la differenza di scala tra le variabili potrebbe causare problemi nel calcolo della distanza stessa (le recensioni sono nell'ordine delle migliaia, mentre la RAM ha valori compresi tra 0 e 16). Per evitare questi problemi, lo `StandardScaler()` viene applicato in modalità selettiva sulle sole variabili continue, calcolando media e varianza sul solo set di train e propagandole sul test set, preservando l'isolamento metrico delle dummy categoriali. La Grid Search seleziona dinamicamente il valore di *K* geometricamente più stabile.

4.2 Logistic Regression

La scelta della Regressione Logistica permette di stabilire una *baseline* probabilistica per la classificazione binaria del successo. Al fine di calibrare la forza della regolarizzazione e prevenire l'overfitting nello scenario esteso, la Grid Search esplora diversi valori per l'iperparametro di penalità C: [0.1, 1.0, 10.0]. Il parametro `max_iter=1000` garantisce la convergenza dell'ottimizzatore numerico anche nello scenario esteso (+KB), dove la dimensionalità aumenta a causa del *One-Hot Encoding*. Anche in questo caso, per consentire una corretta regolarizzazione dei coefficienti e ottimizzare la discesa del gradiente, l'addestramento e la successiva fase di `predict` vengono eseguiti sulla matrice dei dati precedentemente normalizzata tramite lo `StandardScaler`.

4.3 Decision Tree

Il Decision Tree introduce una logica di classificazione basata su regole di split geometriche. In questo modello, la Grid Search opera un bilanciamento strutturale ottimizzando la profondità massima dell'albero (`max_depth`: [3, 5, 10]) e il criterio di purezza del nodo (`criterion`: ['gini', 'entropy']) per prevenire fenomeni di *overfitting*. A differenza dei precedenti, il *Decision Tree* non richiede la standardizzazione dei dati poiché gli split sui nodi sono invarianti rispetto alle trasformazioni di scala monotone. L'algoritmo viene quindi addestrato sulle feature originali (`X_train_r` e `X_train_k`). L'iperparametro ottimale selezionato dalla ricerca sulla griglia forza una regolarizzazione strutturale, rendendo l'albero un ottimo stimatore per valutare l'impatto diretto della feature discreta *High_Hype* nei primi livelli di ramificazione.

4.4 Random Forest Classifier

Si tratta di un modello *ensemble* basato sul principio del *bagging* (aggregazione di alberi decisionali indipendenti). Lo spazio degli iperparametri mappato dalla Grid Search valuta combinazioni sul numero di stimatori paralleli (`n_estimators`: [50, 100]) e sui limiti di crescita (`max_depth`: [5, 10, None]). Il *Random Forest* è stato selezionato per evidenziare le interazioni complesse introdotte dall'ontologia (come l'incrocio tra la fascia di prezzo e il rischio commerciale). Sfruttando il campionamento casuale sia dei record che delle feature, il modello mitiga intrinsecamente la varianza del singolo albero. Come per il Decision Tree, l'addestramento non necessita di scaling metrico.

4.5 Analisi degli iperparametri ottimali selezionati

La fase di *Fine-Tuning*, attraverso l'uso del `GridSearchCV`, ha selezionato gli stessi iperparametri ottimali sia per lo scenario grezzo sia per lo scenario arricchito (+KB). Per il modello *K-NN*, la

griglia ha stabilito che impostare il calcolo su soli 3 vicini (`n_neighbors = 3`) offre confini di decisione locali molto più precisi. La *Logistic Regression* ha preferito una penalità `C = 10.0`, che permette ai coefficienti delle feature (specialmente quelle introdotte dall'ontologia) di esprimere tutto il loro peso matematico. Infine, i modelli basati su alberi lavorano al meglio spingendosi in profondità: il *Decision Tree* utilizza una ramificazione a 10 livelli (`max_depth = 10`) con indice di Gini, mentre il *Random Forest* ottiene il massimo delle performance nel momento in cui è senza limiti strutturali (`max_depth = None`) su una base di 100 stimatori. Questa stabilità dimostra che l'incremento delle metriche deriva esclusivamente nel puro valore informativo aggiunto dalla base di conoscenza semantica.

```
-> Avvio della simulazione su 10 Run indipendenti con Grid Search e Scaler Selettivo...
[Run 1] K-Nearest Neighbors (K-NN) (Grezzo) - Parametri migliori: {'n_neighbors': 3}
[Run 1] K-Nearest Neighbors (K-NN) (+KB) - Parametri migliori: {'n_neighbors': 3}
[Run 1] Logistic Regression (Grezzo) - Parametri migliori: {'C': 10.0}
[Run 1] Logistic Regression (+KB) - Parametri migliori: {'C': 10.0}
[Run 1] Decision Tree Classifier (Grezzo) - Parametri migliori: {'criterion': 'gini', 'max_depth': 10}
[Run 1] Decision Tree Classifier (+KB) - Parametri migliori: {'criterion': 'gini', 'max_depth': 10}
[Run 1] Random Forest Classifier (Grezzo) - Parametri migliori: {'max_depth': None, 'n_estimators': 100}
[Run 1] Random Forest Classifier (+KB) - Parametri migliori: {'max_depth': None, 'n_estimators': 100}
```

Figure 6: Parametri alla prima run

4.6 Risultati ottenuti

Il confronto tra lo Scenario A (*Raw*) e lo Scenario B (*+KB*) mostra un aumento significativo di tutte le metriche di performance (*Accuracy*, *Precision* e *F1-Score*) su tutti i modelli addestrati, evidenziando l'efficacia del connubio tra sintonizzazione iperparametrica automatica e iniezione semantica.

La ridotta deviazione standard ($\pm\sigma$) registrata nei run dimostra la stabilità dei modelli, escludendo un miglioramento dovuto a casualità nel campionamento indotto da `train_test_split`. Questa omogeneità dei risultati funge da indicatore di bassa instabilità, confermando che l'introduzione della Cross-Validation interna nella Grid Search ha efficacemente sterilizzato i modelli dal rischio di overfitting.

L'analisi del comportamento permette di trarre particolari conclusioni:

- **K-NN:** Qui è presente il salto prestazionale più netto, in particolare sull'*F1-Score*. La natura geometrica del K-NN faticava a separare le classi a causa della distribuzione fortemente asimmetrica e schiacciata delle variabili grezze (come il volume delle recensioni). L'introduzione delle feature dall'ontologia (es. il profilo prestazionale `Hardware_Tier` e la classe di rischio `Commercial_Risk`), opportunamente isolate dallo scaling lineare, ha ridefinito lo spazio, creando cluster nettamente più densi e polarizzati rispetto al target.
- **Regressione Logistica:** Anche in questo caso, le feature estratte dall'ontologia hanno portato risultati migliori. Nello Scenario B, la combinazione lineare delle probabilità viene arricchita da predittori discreti forti e stabilizzata dal coefficiente `C` ottimale trovato dalla Grid Search, riducendo il tasso di falsi positivi (conseguente incremento della *Precision*).
- **Decision Tree e Random Forest:** Entrambi gli algoritmi basati su alberi decisionali si confermano i migliori stimatori del sistema. Il *Decision Tree*, sintonizzato dinamicamente sui parametri di split e profondità, sfrutta la feature `High_Hype` per eseguire i primi sezionamenti ad alta entropia, isolando grandi porzioni di record positivi già nei nodi primari della struttura. Il *Random Forest* beneficia del meccanismo di *bagging* e della selezione iperparametrica ottima: l'alta dimensionalità introdotta dal *One-Hot Encoding* permette alla foresta di catturare interazioni non lineari profonde (es. la correlazione tra un prezzo premium e un basso volume di recensioni codificata nel rischio commerciale) spingendo l'accuratezza generale verso i massimi livelli registrati nel progetto.

In conclusione, questi risultati dimostrano che l'arricchimento del dataset supera i limiti dell'apprendimento sui soli dati grezzi. L'estrazione di informazioni tramite query SPARQL e la loro successiva introduzione nel flusso forniscono ai modelli un contesto logico (il "significato" dei dati); grazie a questa robusta infrastruttura, supportata da una sistematica ricerca a griglia degli iperparametri, il sistema non si

limita a interpolare curve numeriche, ma apprende relazioni causali e di dominio stabili, massimizzando l'efficacia predittiva della pipeline.

	Modello	Accuratezza (Grezzo)	Accuratezza (+KB)	F1-Score (Grezzo)	F1-Score (+KB)
0	K-Nearest Neighbors (K-NN)	92.97% $\pm$ 0.31	90.74% $\pm$ 0.37	91.80% $\pm$ 0.39	89.21% $\pm$ 0.45
1	Logistic Regression	82.11% $\pm$ 0.64	82.17% $\pm$ 0.58	74.05% $\pm$ 1.15	74.14% $\pm$ 1.05
2	Decision Tree Classifier	97.52% $\pm$ 0.15	97.52% $\pm$ 0.15	97.16% $\pm$ 0.17	97.16% $\pm$ 0.17
3	Random Forest Classifier	99.28% $\pm$ 0.08	98.22% $\pm$ 0.16	99.17% $\pm$ 0.09	97.96% $\pm$ 0.18

Figure 7: Tabella di confronto dei 4 modelli utilizzati

5 Rete bayesiana

Nelle fasi precedenti il sistema è stato addestrato a eseguire una pulizia deterministica, a estrarre regole logiche rigide dall'ontologia e a formulare predizioni nette tramite modelli di Machine Learning. Nella realtà del mercato di Steam, tuttavia, le dinamiche commerciali sono raramente riconducibili a scenari precisi: le decisioni strategiche si muovono quasi sempre in contesti dominati da una forte componente di incertezza, dove fattori diversi si influenzano a vicenda.

Per catturare questa complessità, è stata implementata una **Rete Bayesiana** (sfruttando la libreria `pgmpy`), ovvero un modello probabilistico grafico orientato che permette di superare i limiti della logica booleana classica attraverso due elementi principali:

1. **Modellazione causale della struttura (DAG)**: viene definito un grafo aciclico diretto (*Directed Acyclic Graph*). La struttura del grafo riflette le relazioni di causa-effetto stabilite a priori dal dominio: il modello di business (`Monetization_Model`) e i requisiti tecnici (`Hardware_Tier`) determinano congiuntamente il livello di rischio commerciale del titolo (`Commercial_Risk`); a loro volta, il rischio finanziario e l'impatto di popolarità (`High_Hype`) convergono per determinare la probabilità condizionata della variabile obiettivo finale (`Target_Success`).
2. **Stima dei parametri e inferenza esatta**: Le tabelle di probabilità condizionata (*CPT*) vengono calibrate sui dati reali del dataset sfruttando lo stimatore di massima verosimiglianza `MaximumLikelihoodEstimator`. Una volta consolidata la base di conoscenza probabilistica, viene configurato il motore di inferenza esatta `VariableElimination` (Eliminazione di Variabile).

Questo motore permette di interrogare la rete formulando query probabilistiche su scenari ipotetici. Impostando determinate condizioni di partenza come vincoli fissi (le "evidenze"), il sistema calcola la probabilità posteriore del `Target_Success`. Nello specifico, il codice mette alla prova la rete simulando quattro archetipi di gioco tipici di Steam (dal grande Blockbuster Premium ad alto budget fino al fenomeno Free-To-Play leggero), dimostrando come il profilo contestuale alteri radicalmente le distribuzioni di probabilità del successo commerciale.

5.1 DAG

Si inizia dalla preparazione del dataset `df_bayes`, in cui tutte le variabili continue o discrete vengono forzate a stringhe testuali (es. '0' e '1' per i flag binari). Questa operazione è necessaria perché la libreria `pgmpy` richiede categorie discrete e ben definite per poter calcolare le frequenze.

L'elemento fondamentale della rete è rappresentata dal Grafo Aciclico Diretto (*DAG*). Questa topologia esprime visivamente le dipendenze causali strutturate:

- Il modello di monetizzazione (`Monetization_Model`) e la richiesta hardware (`Hardware_Tier`) sono i nodi padri originari. Essi determinano il livello di rischio finanziario del videogioco (`Commercial_Risk`).
- A sua volta, il rischio calcolato si unisce all'impatto virale del titolo sul pubblico (`High_Hype`). Entrambi i nodi puntano alla variabile foglia finale, ovvero il `Target_Success`.

5.2 CPT

Una volta definito il DAG, il sistema deve "riempire" i nodi con i numeri reali. Questa operazione viene effettuata tramite lo stimatore di massima verosimiglianza `MaximumLikelihoodEstimator` che analizza le circa 30.000 righe del dataset e calcola le *Tabelle di Probabilità Condizionata* (CPT). Per i nodi radice (come il modello di business), la CPT conterrà semplicemente le probabilità a priori del mercato (es. la percentuale complessiva di giochi gratuiti rispetto a quelli a pagamento). Per i nodi figli (come `Commercial_Risk`), la CPT memorizzerà invece una matrice di probabilità condizionate, calcolando la frequenza del rischio per ogni singola combinazione dei nodi genitori.

5.3 Simulazione degli scenari

L'algoritmo di *Eliminazione di Variabile* risolve le equazioni probabilistiche della rete escludendo matematicamente le variabili irrilevanti per la query corrente, riducendo drasticamente i tempi di calcolo.

Vengono simulati quattro scenari commerciali differenti, analizzando come l'introduzione di un'evidenza (*evidence*) modifichi la distribuzione di probabilità posteriore del successo del gioco.

5.3.1 Il blockbuster acclamato

Nel primo scenario viene testato il profilo di un grande titolo a pagamento, computazionalmente pesante ma capace di generare un massiccio volume di vendite e un forte Hype di mercato entro una fascia di prezzo competitiva (sotto i 40€). Eseguendo la query probabilistica `exact_engine.query`, la rete rileva che la condizione di `High_Hype = 1` (alta penetrazione commerciale) compensa ampiamente i vincoli restrittivi legati all'hardware high-end. La probabilità posteriore del `Target_Success = 1` raggiunge l'84.83%, dimostrando che l'ottimo posizionamento di prezzo combinato alla risposta di massa del pubblico azzerà l'attrito della barriera hardware.

```

=====
Scenario 1: Il blockbuster acclamato (Tripla A)
=====
Evidenze impostate: {'Monetization_Model': 'Premium_Paid', 'Hardware_Tier': 'High_End_Specs', 'High_Hype': '1'}
+-----+-----+
| Target_Success | phi(Target_Success) |
+-----+-----+
| Target_Success(0) | 0.1517 |
+-----+-----+
| Target_Success(1) | 0.8483 |
+-----+-----+

```

Figure 8: Scenario 1

5.3.2 Flop di critica

Il secondo profilo analizza lo scenario speculare: un titolo premium molto pesante dal punto di vista tecnico, che viene però lanciato a prezzo pieno standard (pari o superiore a 40€) o che non riesce a generare un volume critico di vendite e interazioni immediate nella community (`High_Hype = 0`). In questo caso, l'assenza di Hype commerciale condiziona negativamente la rete, propagando una probabilità condizionata del 58.47% verso lo stato di fallimento commerciale (`Target_Success = 0`). Ciò formalizza matematicamente come un budget di sviluppo elevato non sia una garanzia di successo se il posizionamento di prezzo iniziale non incontra un volume di vendite adatto immediato.

```

=====
Scenario 2: Il tripla A al Lancio (Flop di Critica)
=====
Evidenze impostate: {'Monetization_Model': 'Premium_Paid', 'Hardware_Tier': 'High_End_Specs', 'High_Hype': '0'}
+-----+-----+
| Target_Success | phi(Target_Success) |
+-----+-----+
| Target_Success(0) | 0.5847 |
+-----+-----+
| Target_Success(1) | 0.4153 |
+-----+-----+

```

Figure 9: Scenario 2

5.3.3 Il Free-To-Play leggero

Il terzo scenario modella l'archetipo dei moderni titoli competitivi o cooperativi gratuiti, accessibili su qualsiasi computer e altamente virali:

```
evidence = {'Monetization_Model': 'Free_To_Play',
            'Hardware_Tier': 'Standard_Specs',
            'High_Hype': '1'}
```

Rimuovendo la barriera del prezzo (`Free_To_Play`) e la barriera tecnica dei requisiti hardware (`Standard_Specs`), il modello azzerà il rischio commerciale a monte. L'evidenza concomitante di una community attiva (`High_Hype = 1`) porta la rete a inferire una probabilità di successo estremamente elevata e stabile, tipica dei software scalabili ad alta penetrazione di mercato.

```

=====
Scenario 3: Il fenomeno Free-To-Play leggero (Es. Casual/Esport)
=====
Evidenze impostate: {'Monetization_Model': 'Free_To_Play', 'Hardware_Tier': 'Standard_Specs', 'High_Hype': '1'}
+-----+-----+
| Target_Success | phi(Target_Success) |
+-----+-----+
| Target_Success(0) | 0.1444 |
+-----+-----+
| Target_Success(1) | 0.8556 |
+-----+-----+

```

Figure 10: Scenario 3

5.3.4 L'Indie di nicchia a pagamento

L'ultimo profilo descrive la stragrande maggioranza delle produzioni indipendenti standard: giochi dal prezzo budget o medio, graficamente accessibili ma che non riescono a rompere la barriera della nicchia commerciale per diventare virali:

```
evidence = {'Monetization_Model': 'Premium_Paid',
            'Hardware_Tier': 'Standard_Specs',
            'High_Hype': '0'}
```

In questa simulazione, la rete riscontra un **Commercial_Risk** bilanciato o controllato (grazie ai requisiti hardware leggeri), ma l'assenza di un volume critico di recensioni (**High_Hype** = 0) schiaccia la probabilità del **Target_Success** verso una distribuzione a favore del fallimento o della stagnazione delle vendite.

```

=====
Scenario 4: L'indie di nicchia a pagamento (grafica semplice)
=====
Evidenze impostate: {'Monetization_Model': 'Premium_Paid', 'Hardware_Tier': 'Standard_Specs', 'High_Hype': '0'}
+-----+-----+
| Target_Success | phi(Target_Success) |
+-----+-----+
| Target_Success(0) | 0.5842 |
+-----+-----+
| Target_Success(1) | 0.4158 |
+-----+-----+

```

Figure 11: Scenario 4

5.4 Inferenza diagnostica inversa

Sfruttando il motore di calcolo esatto, è possibile effettuare il cosiddetto **ragionamento diagnostico inverso**: impostando lo stato finale del mercato come vincolo fisso (evidenza), si risale lungo le frecce del grafo causale per analizzare ex-post cosa sia successo a monte.

5.4.1 Profilo di rischio dei giochi di successo

La prima query chiede alla rete: *"Se sappiamo già che un gioco è stato un successo reale (**Target_Success** = 1), qual era il suo livello di rischio commerciale preventivo?"*. L'output della rete restituisce una probabilità netta:

- **Commercial_Risk(Rischio_Controllato)**: 98.62%
- **Commercial_Risk(Rischio_Elevato)**: 1.38%

Questo dato dimostra la coerenza strutturale del sistema: la maggior parte dei titoli che raggiungono il successo presenta un profilo di rischio precedentemente mitigato e controllato (es. corretta calibrazione del prezzo in base al genere). Solo una piccolissima nicchia dell'1.38% riesce a sfondare partendo da una condizione finanziaria iniziale considerata pericolosa.


```
-> Profilo di RISCHIO COMMERCIALE dei giochi che hanno avuto SUCCESSO:
```

Commercial_Risk	phi(Commercial_Risk)
Commercial_Risk(Rischio_Controllato)	0.9862
Commercial_Risk(Rischio_Elevato)	0.0138

Figure 12: Risultati ottenuti

5.4.2 Probabilità di Hype tra i fallimenti commerciali

La seconda query analizza la controparte dei prodotti vincenti, ovvero i titoli che hanno registrato un flop (`Target_Success = 0`), per capire se a questi software mancasse solo il voto della critica o la spinta del pubblico:

- `High_Hype(0)`: 98.96%
- `High_Hype(1)`: 1.04%

La rete quantifica che nel 98.96% dei casi, il fallimento su Steam coincide matematicamente con la totale assenza di Hype commerciale (ossia pochi volumi di recensioni e scarsa penetrazione visiva). Al contrario, esiste un marginale 1.04% di prodotti che, pur avendo generato un forte Hype e vendite rapide iniziali, finisce comunque per floppare nel lungo periodo (ad esempio a causa di bug critici o un punteggio di gradimento globale che crolla sotto la soglia di stabilità).

```
-> Probabilità di avere HIGH HYPE tra i giochi che hanno FLOPPATO:
```

High_Hype	phi(High_Hype)
High_Hype(0)	0.9896
High_Hype(1)	0.0104

Figure 13: Risultati ottenuti

5.4.3 Impatto dell'hardware sul rischio

In questo terzo caso si sperimenta il **ragionamento intermedio**. Isolando esclusivamente i nodi superiori del grafo, si chiede alla rete se richiedere un computer molto potente (`Hardware_Tier = High_End_Specs`) implichi automaticamente un suicidio finanziario a prescindere dal successo del gioco:

- `Commercial_Risk(Rischio_Controllato)`: 97.82%
- `Commercial_Risk(Rischio_Elevato)`: 2.18%

La risposta della rete evidenzia che i requisiti tecnici elevati pesano negativamente sulle finanze solo nel 2.18% dei casi. Nel restante 97.82%, i titoli graficamente pesanti mantengono un rischio controllato, poiché gli editori applicano strategie di bilanciamento (come l'inserimento in generi storicamente molto redditizi) per ammortizzare i costi di sviluppo.

```
-> Distribuzione del RISCHIO COMMERCIALE per i soli titoli HARDWARE HIGH-END:
```

Commercial_Risk	phi(Commercial_Risk)
Commercial_Risk(Rischio_Controllato)	0.9782
Commercial_Risk(Rischio_Elevato)	0.0218

Figure 14: Risultati ottenuti

5.4.4 Il caso del Free-To-Play che fallisce

L'ultimo test sfrutta il principio dell'anomalia di mercato (*Explaining Away*). I giochi gratuiti (**Free-To-Play**) partono avvantaggiati perché azzerano il rischio del prezzo per l'utente. Ci si chiede: "Se un titolo gratuito fallisce, cosa è andato storto?":

- High_Hype(0): 98.43%
- High_Hype(1): 1.57%

Il motore diagnostico individua la causa primaria nel 98.43% dei casi: un gioco gratuito muore se fallisce nel creare una community attiva e un Hype di massa fin dai primi giorni dal lancio. La gratuità del software non serve a nulla se il prodotto rimane invisibile nel marketplace. Il restante 1.57% rappresenta invece i titoli che, pur avendo generato Hype e download massicci all'inizio, falliscono a causa di un pessimo supporto post-lancio che ne distrugge la reputazione.

```
-> Probabilità di HIGH HYPE per un titolo FREE-TO-PLAY che ha FLOPPATO:
```

High_Hype	phi(High_Hype)
High_Hype(0)	0.9843
High_Hype(1)	0.0157

Figure 15: Risultati ottenuti

5.4.5 Conclusioni

I quattro test dimostrano la flessibilità della Rete Bayesiana rispetto ai modelli di Machine Learning tradizionali: mentre un classificatore standard si limita a restituire un'etichetta rigida, la rete fornisce una *misura dell'incertezza* sotto forma di distribuzione probabilistica condizionata, fungendo da perfetto strumento di simulazione strategica per l'analisi predittiva del mercato videoludico.

L'introduzione delle query di inferenza diagnostica inversa e intermedia ha permesso di sbloccare una comprensione più profonda delle dinamiche economiche di Steam. La rete non si limita a dire se un prodotto vincerà o perderà, ma è in grado di navigare le relazioni di causa-effetto. Come dimostrato dagli esperimenti, il modello riesce a quantificare matematicamente i fattori di rischio strutturali, a isolare le cause scatenanti dietro i fallimenti dei prodotti nativamente avvantaggiati (come i titoli gratuiti) e a mappare la composizione dei profili vincenti.

6 Conclusioni e sviluppi futuri

Il progetto mostra come l'integrazione dell'ingegneria della conoscenza e le tecniche di analisi del mercato abbiano avuto risultati notevoli. Partendo da un dataset industriale grezzo e affetto da forti lacune informative, la pipeline sviluppata è riuscita a trasformare il dato atomico in conoscenza strutturata. L'introduzione di un'ontologia astratta in formato **RDF/XML**, interrogata tramite query **SPARQL**, ha permesso di implementare un motore a regole in grado di sanare i valori mancanti attraverso un'imputazione semantica e simmetrica. I modelli di Machine Learning supervisionato hanno fatto registrare incrementi netti nelle metriche di *Accuracy* e *F1-Score* grazie all'iniezione delle feature logiche, mentre la Rete Bayesiana ha fornito uno strumento per simulare scenari ed effettuare decision-making probabilistico in contesti dominati dall'incertezza.

Considerando una futura evoluzione, questi sono state le idee considerate:

1. **Estensione del grafo semantico:** Attualmente l'ontologia mappa le relazioni di sfondo a livello di categorie e generi (*TBox*). Un naturale sviluppo futuro consiste nel mappare l'intero dataset da 30.000 righe in triple RDF concrete, popolando interamente la *ABox* all'interno di un *Triple Store* (come GraphDB o Apache Jena). Questo permetterebbe di effettuare query grafiche complesse e di collegare l'ontologia di Steam a basi di conoscenza esterne e globali come **DBpedia** o **Wikidata** (meccanismo di *Linked Data*), arricchendo i giochi con informazioni storiche sui publisher o premi vinti.
2. **Evoluzione dal Machine Learning:** Lo sviluppo futuro prevede l'addestramento di reti neurali a grafo (**GCN - Graph Convolutional Networks**). In questo modo, i modelli di classificazione apprenderebbero direttamente dalla topologia e dalle relazioni geometriche del grafo RDF.
3. **Apprendimento strutturale della Rete Bayesiana:** Uno sviluppo futuro prevede l'implementazione di algoritmi di *Structure Learning* (basati su vincoli o su punteggi, come il *Hill-Climb Search*). Questo permetterebbe al sistema di scoprire in modo autonomo, analizzando i dati, relazioni nascoste tra le variabili, che le euristiche standard potrebbero non cogliere.